Tariff Switch Request Processor
Technical Documentation & User Guide (Concise)

Build target: .NET 10 (C#) | Date: 2026-02-18

1. Purpose

A lightweight .NET console application that processes customer tariff-switch requests from CSV files. For each request the system applies validation and business rules to produce an approval or rejection decision, computes an SLA due date in the Europe/Vienna timezone (DST-safe), and persists follow-up actions and processed request IDs to allow idempotent reruns.

2. Architecture Overview

The application follows a pipeline-style architecture with clear separation between input validation, lookup loading, request processing, SLA calculation, output writing, and persistence of processed request IDs. All processing is local and file-based (CSV).

- Resolve input directory (default ./inputFiles or CLI argument).
- Fail-fast validation of folder, required files, and CSV headers.
- Load processed request IDs into an in-memory HashSet.
- Load customers and tariffs into dictionaries for fast lookup.
- Stream requests row-by-row; skip processed IDs; apply business rules.
- Compute SLA deadlines in Europe/Vienna local time with DST-safe handling.
- Write outputs and persist processed request IDs.

3. Components and Responsibilities

- Program.cs — Entry point that validates arguments and runs RequestProcessor.
- InputPathResolver.cs — Resolves the input directory.
- InputPathValidator.cs — Validates folder, required files, and CSV schema.
- InputFiles.cs — Stores paths to customers, tariffs, and requests files.
- CsvHeaderMap.cs — Handles case-insensitive header mapping.
- CsvLookupLoader.cs — Loads customers and tariffs into dictionaries.
- RequestProcessor.cs — Main orchestration and rule execution.
- SlaCalculator.cs — Computes SLA deadlines in Europe/Vienna with DST handling.
- DecisionWriter.cs — Writes decisions.csv and followups.csv.
- ProcessedRequestStore.cs — Persists processed request IDs for idempotency.

4. Input, Validation, and Error Handling

Configuration errors such as missing folders, files, or headers cause the application to fail fast with a descriptive message.

Row-level data errors do not stop processing. Instead, the affected request is rejected with a reason and processing continues.

5. Business Rules

- Reject request if data is invalid or timestamp cannot be parsed.
- Reject request if customer ID is unknown.
- Reject request if tariff ID is unknown.
- Reject request if customer has unpaid invoices.
- Approve request otherwise and compute SLA deadline.
- If tariff requires a smart meter and the customer does not have one: approve request, create follow-up action 'Schedule meter upgrade', and extend SLA accordingly.

6. Idempotency

Processed request IDs are stored in ./processedRequestIDs/processed_ids.csv. At startup the file is loaded into an in-memory HashSet to allow constant-time membership checks. Already processed requests are skipped.

7. SLA and Timezone Handling

SLA deadlines are calculated using Europe/Vienna local time semantics rather than UTC arithmetic to ensure correctness across daylight-saving transitions.

- Premium SLA: +24 hours
- Standard SLA: +48 hours
- Smart meter upgrade lead time: +12 hours
- Follow-up deadline: SLA due minus 12 hours

8. Outputs

Output files are written to ./outputFiles. Each run recreates the files with headers to ensure deterministic results.

- decisions.csv → RequestId;Decision;Reason;SlaDueISO8601
- followups.csv → RequestId;Action;DeadlineISO8601
- Rejected requests use 'N/A' for the SLA due column.